1 η Εργασία στη Σχεδίαση ΒΔ 2025-2026 Ευρετήρια και Βελτιστοποίηση Επερωτήσεων

Ζόγκας Αλέξανδρος
it2023017

<u>1 ο Ερώτημα: Ευρετήριο με Β+ δέντρο [65 μονάδες]</u>

Ένα αρχείο σωρού έχει 2.000.000 εγγραφές, αποθηκευμένες με μη-εκτεινόμενη καταχώρηση. Η κάθε εγγραφή έχει (σταθερό) μέγεθος 1000 bytes και το κάθε block έχει μέγεθος 4096 bytes.

1) Ποιος είναι ο παράγοντας ομαδοποίησης (blocking factor); [5 μονάδες]

$$r = 2.000.000 \text{ εγγραφές}$$
$$R = 1000\ bytes \text{ η μία εγγραφή}$$
$$B = 4096\ bytes \text{ μέγεθος } block$$

$$bfr = \frac{4096}{1000} = 4 \text{ εγγραφές σε κάθε } block$$

2) Πόσα byte μένουν αναξιοποίητα σε κάθε block, λόγω της μη εκτεινόμενης καταχώρησης; [5 μονάδες]

Αφού κάθε $block$ χωράει 4 εγγραφές και η κάθε εγγραφή θέλει 1000 $bytes$ τότε:

$$4 \times 1000 = 4000\ bytes$$

άρα

$$4096 - 4000 = 96\ bytes \text{ μένουν αναξιοποίητα σε κάθε } block$$

3) Πόσα block απαιτούνται για την αποθήκευση ολόκληρου του αρχείου; [5 μονάδες]

$$Sf = \frac{2.000.000}{4} = 500.000\ blocks$$

4) Αν το κόστος μεταφοράς ενός block από/προς το δίσκο είναι 0,05ms, [10 μονάδες]

4.α) Ποιο είναι το κόστος (χρόνος) αναζήτησης με συνθήκη ισότητας;

$$\text{Κόστος Ισότητας} = 0.5 \times \text{B} \times D = 0,5 \times 500.000 \times 0,05\,ms = 12.500$$

4.β) Ποιο το κόστος (χρόνος) αναζήτησης με συνθήκη εύρους;

$$\text{Κόστος Εύρου} = B \times D = 500.000 \times 0.05 = 25.000$$

5) Πάνω σε ένα πεδίο char(20) που μπορεί και να περιέχει διπλότυπα, έχει χτιστεί ευρετήριο με χρήση B+ δέντρου. Κάθε δείκτης προς τις εγγραφές του αρχείου έχει μέγεθος 11 bytes, κάθε δείκτης προς block του ευρετηρίου, όπως και ο κάθε δείκτης προς επόμενο φύλλο, έχει μέγεθος 16 bytes.

Δεδομένα

Μέγεθος $block = 4096\ bytes$

Πεδίο $(key) = char(20)$ άρα $20\ bytes$

Δείκτης προς εγγραφή $= 11\ bytes$

Δείκτης προς $block$ του ευρετηρίου $= 16\ bytes$

Δείκτης προς επόμενο φύλλο $= 16\ bytes$

5α) Ποια είναι η τάξη και ποιο το fanout του B+ δέντρου, ώστε κάθε κόμβος-φύλλο να είναι όσο το δυνατόν γεμάτος; [15 μονάδες]

Υπολογισμός της χωρητικότητας των φύλλων:

Κάθε ζεύγος στο φύλλο έχει:

Key: $20\ bytes$

$Pointer$ προς την εγγραφή: $11\ bytes$

Άρα το μέγεθος ανά στοιχείο στο φύλλο είναι:

$20 + 11 = 31\ bytes$

Κάθε φύλλο έχει επίσης και έναν $pointer$ προς το επόμενο φύλλο: $16\ bytes$

Οπότε η μέγιστη χωρητικότητα: k είναι οι εγγραφές

$31k + 16 \leq 4096$

$31k \leq 4080$

$k \leq 131.6$

Άρα $k = 131$ εγγραφές ανά φύλλο

Υπολογισμός χωρητικότητας εσωτερικών κόμβων ($fanout$)

Κάθε εσωτερικός κόμβος έχει:

Key: 20 $bytes$

$Pointer$ προς $child\ block$: 16 $bytes$

Σε έναν εσωτερικό κόμβο με k κλειδιά υπάρχουν $k + 1$ δείκτες.

Άρα το συνολικό μέγεθος:

$$(20k + 16(k + 1)) \leq 4096$$
$$(36k + 16) \leq 4096$$
$$36k \leq 4080$$
$$k \leq 113,3$$

Άρα $k + 1 = 114 = fanout$

Υπολογισμός "τάξης" του B + δέντρου

Η τάξη του δέντρου $d = \frac{fanout}{2} = \frac{114}{2} = 57$

5β) Πόσα επίπεδα υπάρχουν στο B+ δέντρο (συμπεριλαμβανομένου και του τελευταίου επιπέδου) και πόσοι κόμβοι ανά επίπεδο; Θεωρήστε ότι οι εσωτερικοί κόμβοι είναι γεμάτοι κατά τα 2/3. [10 μονάδες]

$Sφύλλα = \frac{2.000.000}{131} = 15.267,9$ άρα 15.268 φύλλα

– Είναι γεμάτο κατά τα $2/3$

Άρα το $fanout = 114 \times \frac{2}{3} = 76$

– Δηλαδή, κάθε εσωτερικός κόμβος δείχνει περίπου σε 76 παιδιά.

Άρα Αριθμός κόμβου επόμενου επιπέδου $= \frac{15.268}{76} = 201$ κόμβους

Άρα Υπολογισμός κόμβων του επόμενου επιπέδου $= \frac{201}{76} = 2,64$ άρα 3 κόμβοι

Άρα Υπολογισμός ρίζας ($root$) $= \frac{3}{76} = 0,04 = 1$ κόμβος

5γ) Ποιο είναι το μέγεθος (σε block) του ευρετηρίου συνολικά; [5 μονάδες]

Σύνολο $blocks = 1 + 3 + 201 + 15.268 = 15.473\ blocks$

5δ) Ποιο θα είναι το κόστος (χρόνος) αναζήτησης για κλειδιά με τιμή (για το πεδίο αναζήτησης) μικρότερη ή ίση κάποιας τιμής που εμφανίζεται για τελευταία φορά στο τέταρτο φύλλο; Θεωρήστε και πάλι ότι το κόστος μεταφοράς ενός block από/προς το δίσκο είναι 0,05ms. [10 μονάδες]

Κατεβαίνουμε από τη ρίζα μέχρι το κατάλληλο φύλλο.

Το ύψος του δέντρου από το 5β ήταν:

Επίπεδο 1 → Ρίζα

Επίπεδο 2 → 3 κόμβοι

Επίπεδο 3 → 201 κόμβοι

Επίπεδο 4 → 15. 268 φύλλα

Άρα ύψος $=$ 4 επίπεδα $=$ άρα 4 $block\ accesses$ μέχρι το πρώτο φύλλο

Και επειδή διαβάζουμε μέχρι το 4 φύλλο τότε έχουμε συνολικό πλήθος $block\ accesses\ \ 4 + 4 = 8$

Άρα το Κόστος αναζήτησης είναι $0.05 \times 8 = 0.40ms$

<u>2 ο Ερώτημα: Υλοποίηση με JDBC [35 μονάδες]</u>
Θα χρησιμοποιήσουμε τη ΒΔ που φτιάξατε στο μάθημα «Βάσεις Δεδομένων», η οποία διατηρούσε στοιχεία υπαλλήλων, τα έργα και τις τοποθεσίες στα οποία απασχολούνται, εξαρτώμενα μέλη κτλ.

Η δομή του κώδικα:

```
src/
 └── main/
      └── java/
           └── org/example/
                ├── Main.java
                ├── Menu.java
                │
                ├── db/
                │    └── DBUtils.java
                │
                ├── model/
                │    └── Employee.java
                │
                ├── services/
                │    ├── Queries/
                │    │    └── DepartmentQueries.java
                │    │
                │    ├── CreateEmployeeIndex.java
                │    ├── EmployeeInserter.java
                │    ├── EmployeeQueries.java
                │    └── ExecutionPlanPrinter.java
                │
                └── utils/
                     └── RandomDataGeneration.java
```

Ανάλυση Κλάσεων

- **Main.java**

Είναι η βασική κλάση της εφαρμογής. Από εδώ ξεκινάει το πρόγραμμα και δημιουργείται το αντικείμενο Menu, το οποίο χειρίζεται τις επιλογές του χρήστη.

- **Menu.java**

Περιλαμβάνει το μενού που εμφανίζεται στον χρήστη και επιτρέπει την επιλογή των λειτουργιών της εφαρμογής.

```
====== ΕΠΙΛΟΓΕΣ ======
1) Εισαγωγή 2.000.000 υπαλλήλων.
2) Προβολή ονομάτων, επωνύμων ,μισθών και τοποθεσία τμήματος.
3) Προβολή πλάνου εκτέλεσης.
4) Δημιουργία ευρετηρίου.
0) Έξοδός.
Επέλεξε από 0 εώς 4:
```

*2.000.000 λάθος είναι 100.000

- **DBUtils.java**

Αναλαμβάνει τη σύνδεση με τη βάση δεδομένων.

- **Employee.java**

Αναπαριστά έναν υπάλληλο όπως υπάρχει στη βάση. Βοηθάει στη δημιουργία αντικειμένων πριν εισαχθούν στη βάση.

- **DepartmentQueries.java**

Περιέχει SQL ερωτήματα που αφορούν τα τμήματα και χρησιμοποιείται για να λαμβάνονται τιμές όπως dno και mgrssn.

- **CreateEmployeeIndex.java**

Υλοποιεί τη δημιουργία του index που ζητείται στο αντίστοιχο ερώτημα.

- **EmployeeInserter.java**

Εισάγει τους ψεύτικους υπαλλήλους στη βάση δεδομένων χρησιμοποιώντας τα τυχαία δεδομένα που παράγονται.

- **EmployeeQueries.java**

Εκτελεί το ερώτημα του δεύτερου σκέλους της άσκησης και ανακτά SSN ώστε να μην παραχθούν διπλότυπα.

- **ExecutionPlanPrinter.java**

Εκτελεί το ερώτημα του τρίτου σκέλους και εμφανίζει το πλάνο εκτέλεσης με EXPLAIN.

- **RandomDataGeneration.java**

Παράγει τα τυχαία δεδομένα των υπαλλήλων, όπως ονόματα, SSN και μισθούς.

1) Φτιάξτε ένα πρόγραμμα Java που να εισάγει 2.000.000 υπαλλήλους. Τα στοιχεία τους να είναι τυχαία, αλλά θα πρέπει να ικανοποιούν τους περιορισμούς ακεραιότητας. [10 μονάδες]

(Σε αυτό το ερώτημα έχω εισάγει μόνο 100.000 εγγραφές καθώς δεν με άφηνε να εισάγω παραπάνω.)

Έφτιαξα την κλάση **EmployeeInserter** που αναλαμβάνει να εισάγει στη βάση δεδομένων τους ψεύτικους υπαλλήλους που δημιουργούνται.
Κατά την αρχικοποίησή της, διαβάζει από τη βάση όλα τα SSN των υπαλλήλων, τα SSN των προϊσταμένων και όλα τα διαθέσιμα τμήματα *. Αυτές οι πληροφορίες περνιούνται στη `RandomDataGeneration`, η οποία χρησιμοποιείται για να δημιουργήσει ένα μεγάλο σύνολο έγκυρων ψεύτικων υπαλλήλων με σωστό `superSSN` ,`ssn` και `dno`. Αφού δημιουργηθούν, η μέθοδος `insertEmployee()` περνάει τους υπαλλήλους έναν-έναν στη βάση εκτελώντας ένα προετοιμασμένο INSERT. Με αυτόν τον τρόπο γίνεται οργανωμένα και με ασφάλεια η εισαγωγή μεγάλου όγκου δεδομένων χωρίς συγκρούσεις SSN της βάσης.

*Λαμβάνει τα δεδομένα αυτά από τις μεθόδους των κλάσεών DepartmentQueries.java και EmployeeQueries.java.

2) Έστω ότι θέλουμε για κάθε υπάλληλο να μας επιστραφεί το επώνυμό του, το όνομά του, καθώς και η τοποθεσία του τμήματος στο οποίο απασχολείται. Εκφράστε το παραπάνω ερώτημα σε SQL και φτιάξτε ένα πρόγραμμα Java που να εκτυπώνει τα αποτελέσματα αυτής της επερώτησης. [5 μονάδες]

Η κλάση **EmployeeQueries** περιέχει το SQL query του ερωτήματος το οποίο ενώνει τον πίνακα **employee** με τον πίνακα **dept_locations** και επιστρέφει το όνομα, το επώνυμο, τον μισθό και την τοποθεσία του τμήματος κάθε υπαλλήλου. Η μέθοδος **fetchEmployeeData()** εκτελεί το συγκεκριμένο ερώτημα, διαβάζει τα αποτελέσματα και τα εμφανίζει σε μορφή πίνακα, με επικεφαλίδες και μορφοποιημένη εκτύπωση.

3) Εκτυπώστε μέσω προγράμματος Java το πλάνο εκτέλεσης της επερώτησης του προηγούμενου βήματος. Εμφανίστε στην αναφορά σας το πλάνο που εκτυπώθηκε

(print screen ή copy&paste) και εξηγήστε κάθε βήμα και κάθε στοιχείο αυτού του πλάνου. [8 μονάδες]

Η κλάση [ExecutionPlanPrinter](#) είναι υπεύθυνη για την εμφάνιση του execution plan ενός SQL ερωτήματος.

```
Execution Plan
Plan hash value: 941328062

-----------------------------------------------------------------------------------------
| Id  | Operation          | Name              | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT   |                   |    16 |   528 |   239   (0)| 00:00:01 |
|*  1 |  HASH JOIN         |                   |    16 |   528 |   239   (0)| 00:00:01 |
|   2 |   INDEX FULL SCAN  | PK_DEPT_LOCATIONS |     5 |    60 |     1   (0)| 00:00:01 |
|   3 |   TABLE ACCESS FULL| EMPLOYEE          |     8 |   168 |   238   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - access("E"."DNO"="DL"."DNUMBER")
```

Τι σημαίνουν οι βασικές στήλες του execution plan

- **Rows:** ο εκτιμώμενος αριθμός γραμμών που θα παραχθούν από το συγκεκριμένο βήμα (στο δικό μου είναι χαμήλο παρα το γεγονος ότι εχω εισάγει 100.000 υπαλληλους) .
- **Bytes:** εκτίμηση του μεγέθους των δεδομένων που θα περάσουν από το βήμα.
- **Cost:** το κόστος που υπολογίζει ο optimizer για τη λειτουργία (όχι χρόνος, αλλά σχετική τιμή).
- **Time:** ο εκτιμώμενος χρόνος που θα χρειαστεί το βήμα.

Ανάλυση βημάτων του πλάνου

- **ID 0 – SELECT STATEMENT**
  Αντιπροσωπεύει το συνολικό ερώτημα. Δεν κάνει κάποια λειτουργία από μόνο του· απλώς συγκεντρώνει το τελικό αποτέλεσμα που παράγεται από τα προηγούμενα βήματα.
- **ID 1 – HASH JOIN**
- Εδώ γίνεται το join των δύο πινάκων. Η Oracle επιλέγει hash join επειδή ο πίνακας *dept_locations* είναι μικρός. Φτιάχνει έναν μικρό hash πίνακα στη μνήμη και τον χρησιμοποιεί για να συνδυάσει γρήγορα όλες τις εγγραφές του *employee*.
- **ID 2 – INDEX FULL SCAN | PK_DEPT_LOCATIONS**
  Η Oracle διαβάζει ολόκληρο το primary key index του *dept_locations*. Ο

πίνακας έχει ελάχιστες γραμμές, οπότε αυτό το βήμα είναι πολύ ελαφρύ και έχει σχεδόν μηδενικό κόστος.

- **ID 3 – TABLE ACCESS FULL | EMPLOYEE**
  Σε αυτό το βήμα σαρώνεται ολόκληρος ο πίνακας *employee*. Επειδή το ερώτημα χρειάζεται όλους τους υπαλλήλους, η Oracle διαβάζει όλες τις γραμμές.

4) Δημιουργήστε μέσω του ίδιου προγράμματος Java ένα κατάλληλο ευρετήριο ώστε η εκτέλεση της παραπάνω επερώτησης να γίνει αποδοτικότερα. Εκτυπώστε ξανά το νέο πλάνο εκτέλεσης της ίδιας επερώτησης. Αιτιολογήστε στην αναφορά σας γιατί επιλέξατε αυτό το ευρετήριο και τι όφελος είχε η δημιουργία του ευρετηρίου αυτού. [12 μονάδες]

```
Execution Plan
Plan hash value: 941328062

-----------------------------------------------------------------------------------------
| Id  | Operation          | Name              | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT   |                   |    16 |   528 |   239   (0)| 00:00:01 |
|*  1 |  HASH JOIN         |                   |    16 |   528 |   239   (0)| 00:00:01 |
|   2 |   INDEX FULL SCAN  | PK_DEPT_LOCATIONS |     5 |    60 |     1   (0)| 00:00:01 |
|   3 |   TABLE ACCESS FULL| EMPLOYEE          |     8 |   168 |   238   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - access("E"."DNO"="DL"."DNUMBER")
Index idx_employee_dno dropped successfully
```

```
SELECT e.fname, e.lname, e.salary, dl.dlocation
FROM employee e
JOIN dept_locations dl ON e.dno = dl.dnumber;
```

```
CREATE INDEX idx_employee_dno ON employee(dno);
```

Δημιούργησα αυτό το ευρετήριο καθώς το **dno** είναι ακριβώς η στήλη που χρησιμοποιείται στη συνθήκη του join (`e.dno = dl.dnumber`) και έτσι ο optimizer έχει πλέον τη

δυνατότητα να χρησιμοποιήσει το index για πιο γρήγορη πρόσβαση στους υπαλλήλους ανά τμήμα. Παρόλα αυτά, το πλάνο εκτέλεσης παρέμεινε το ίδιο με πριν. Αυτό οφείλεται στο γεγονός ότι το συγκεκριμένο ερώτημα χρειάζεται να διαβάσει όλους τους υπαλλήλους, επειδή το αποτέλεσμα περιλαμβάνει κάθε εγγραφή του πίνακα *employee*. Σε μια τέτοι περίπτωση, ο optimizer επιλέγει **TABLE ACCESS FULL**, γιατί είναι πιο γρήγορο να σαρώσει μία φορά ολόκληρο τον πίνακα, αντί να χρησιμοποιήσει το ευρετήριο και στη συνέχεια να προσπελάσει ξανά τον πίνακα για κάθε row. Με άλλα λόγια, το index είναι σωστό και χρήσιμο, αλλά δεν βοηθάει σε αυτό το συγκεκριμένο **SELECT** επειδή δεν υπάρχει περιορισμός (**WHERE** clause) πάνω στο *dno*.

Θα ήταν χρήσιμο αν εκτελούσα queries όπως:

```
WHERE e.dno = 4  ή WHERE e.dno BETWEEN 2 AND 5
```